Secure Private Infrastructure Deployment with Bastion Access and IAM Role-Based Architecture

---

1. Abstract

This project demonstrates the implementation of a **secure and production-ready AWS infrastructure** that follows cloud security best practices. The architecture ensures that application servers are deployed in **private subnets**, eliminating direct internet exposure.

Access to these private instances is controlled via a **bastion host (jump server)** deployed in a public subnet with restricted SSH access. Additionally, the system replaces static credentials with **IAM roles**, ensuring secure and temporary access to AWS services like S3.

The solution highlights **network isolation, controlled access, least privilege principles, and credential-free service interaction**, making it suitable for real-world enterprise environments.

---

2. Introduction

Cloud computing provides scalability and flexibility, but **misconfigurations can lead to severe security risks**, such as:

- Public exposure of sensitive resources
- Hardcoded or leaked credentials
- Unauthorized access to infrastructure

This project focuses on building a **secure AWS infrastructure** that prevents such vulnerabilities by:

- Using **private subnets** for backend systems
- Restricting access via a **bastion host**
- Eliminating static credentials using **IAM roles**

The design follows AWS Well-Architected Framework principles, especially **Security and Reliability pillars**.

---

3. Problem Statement

Many organizations face the following issues:

- EC2 instances deployed with **public IPs**
- SSH access open to the internet (0.0.0.0/0)
- Use of **hardcoded AWS credentials**
- Lack of proper network segmentation

These practices lead to:

- Increased attack surface
- Credential leakage risks
- Non-compliance with security standards

---

4. Objectives

The key objectives of this project are:

1. Design a **secure Virtual Private Cloud (VPC)**
2. Deploy **application servers in private subnets**
3. Implement **bastion host access control**
4. Use **IAM roles instead of static credentials**
5. Enable **secure S3 access from private instances**
6. Follow **least privilege access principle**
7. Validate the security of the architecture

---

5. Technologies Used

- Amazon EC2
- Amazon VPC
- IAM (Identity and Access Management)
- Amazon S3
- NAT Gateway
- Internet Gateway (IGW)
- Nginx Web Server
- AWS CLI

---

6. System Architecture

User → Internet → Internet Gateway → Bastion Host → Private EC2 → IAM Role → S3

Key Components:

- **Internet Gateway (IGW):** Enables internet access for public subnet
- **Bastion Host:** Entry point for SSH access
- **Private EC2:** Application server with no public exposure
- **IAM Role:** Secure access to AWS services
- **S3:** Storage service accessed securely

---

7. Architecture Design Principles

- **Defense in Depth**
- **Zero Trust Networking**
- **Least Privilege Access**
- **Separation of Public and Private Resources**
- **Credential-Free Authentication**

---

8. Network Design

8.1 VPC Configuration

- CIDR Block: 10.0.0.0/16
- DNS Support: Enabled
- DNS Hostnames: Enabled

8.2 Subnet Design

Subnet Type	CIDR	Purpose
Public Subnet	10.0.1.0/24	Bastion Host
Private Subnet	10.0.2.0/24	Application Server

8.3 Route Tables

Public Route Table

- Route: 0.0.0.0/0 → Internet Gateway

Private Route Table

- Route: 0.0.0.0/0 → NAT Gateway

8.4 Internet Gateway (IGW)

- Attached to VPC
- Enables internet access for public subnet

8.5 NAT Gateway

- Placed in public subnet
- Allows private instances to access internet securely
- Prevents inbound internet traffic

---

9. Bastion Host Setup

9.1 Purpose

Acts as a **secure entry point** to access private instances.

9.2 Configuration

- Instance Type: t2.micro (or similar)
- Subnet: Public Subnet
- Public IP: Enabled

9.3 Security Group Rules

Type	Port	Source
SSH	22	Your IP only

9.4 SSH Access Flow

Local Machine → Bastion Host → Private EC2

9.5 SSH Command

ssh -i key.pem ec2-user@bastion-ip

Then:

ssh -i key.pem ec2-user@private-ip

---

10. Private EC2 Instance

10.1 Configuration

- Subnet: Private Subnet
- Public IP: Disabled
- Accessible only via bastion

10.2 Security Group

Type	Port	Source
SSH	22	Bastion SG
HTTP	80	Internal (optional)

10.3 Web Server Setup (Nginx)

```
sudo yum update -y
sudo amazon-linux-extras install nginx1 -y
sudo systemctl start nginx
sudo systemctl enable nginx
```

---

11. IAM Role Implementation

11.1 Purpose

Avoid using:

- Hardcoded credentials

- Access keys stored in code

11.2 Role Creation

- Service: EC2
- Permissions: S3 Read Access

11.3 Policy Example

```
{
  "Effect": "Allow",
  "Action": [
    "s3:ListBucket",
    "s3:GetObject"
  ],
  "Resource": "*"
}
```

11.4 Attach Role to EC2

- Select EC2 instance
- Attach IAM Role

---

12. Secure S3 Access

12.1 Validation Command

aws s3 ls

12.2 Outcome

- Successfully lists buckets
- No credentials required

---

13. Security Validation

13.1 Test Cases

Test	Expected Result
Direct SSH to private EC2	❌ Blocked
SSH via bastion	✅ Allowed
Access S3 without keys	✅ Allowed
Public access to private server	❌ Blocked

---

14. Security Best Practices Implemented

- Private subnets for backend
- Bastion host for controlled access
- IAM roles instead of credentials
- Security groups with restricted access
- NAT Gateway for outbound internet
- No public IPs on private instances

---

15. Advantages of This Architecture

- Strong security posture
- Reduced attack surface
- Scalable and reusable design
- Compliance-ready infrastructure
- Credential-free service interaction

---

16. Limitations

- NAT Gateway cost can be high
- Bastion host requires management
- Manual SSH process (can be improved with SSM)

---

17. Future Enhancements

- Replace bastion with **AWS Systems Manager (SSM)**
- Use **Auto Scaling Groups**
- Implement **Load Balancer**
- Add **CloudWatch monitoring**
- Use **Infrastructure as Code (Terraform/CloudFormation)**

---

18. Real-World Use Cases

- Secure enterprise applications
- Banking and financial systems
- Healthcare systems
- Backend microservices

---

19. Deployment Steps Summary

1. Create VPC
2. Create subnets (public & private)
3. Attach Internet Gateway
4. Create NAT Gateway
5. Configure route tables
6. Launch Bastion Host
7. Launch Private EC2
8. Configure security groups
9. Create IAM role
10. Attach role to EC2
11. Test SSH and S3 access

---

20. Conclusion

This project successfully demonstrates a **secure AWS infrastructure architecture** that eliminates common vulnerabilities such as public exposure and credential misuse.

By implementing:

- **Private networking**
- **Bastion-based access**
- **IAM role authentication**

the system ensures **high security, scalability, and compliance readiness**.